

0484

Bases de datos

Clase 6

06

Programación de bases de datos



Procedimientos y funciones

Los procedimientos almacenados y las funciones definidas por el usuario (UDF) son bloques de código que se guardan en el servidor MySQL y permiten centralizar la lógica de negocio en la base de datos.

Característica	Procedimiento (PROCEDURE)	Función (FUNCTION)
Tipo de retorno	No devuelve valor directamente	Devuelve un valor único obligatorio
Uso en consultas SQL	No se puede usar directamente	Se puede usar dentro de SELECT, WHERE...
Parámetros admitidos	IN, OUT, INOUT	Solo IN
Invocación	CALL nombre()	SELECT nombre()
Aplicación	Automatización de tareas complejas	Cálculo o transformación de un dato

Procedimientos almacenados

Un procedimiento ejecuta instrucciones complejas y puede modificar datos. Son ideales para automatizar procesos. Les podemos pasar parámetros y se ejecutan haciendo un call.

```
-- Procedimiento almacenado. Subir salario de un departamento.  
DELIMITER //  
  
CREATE PROCEDURE subir_salario_departamento(  
    IN p_dep INT,  
    IN p_porcentaje DECIMAL(5,2)  
)  
  
BEGIN  
    UPDATE trabajadores  
    SET salario = salario * (1 + p_porcentaje/100)  
    WHERE numdep = p_dep;  
  
END;  
  
//  
DELIMITER ;  
  
CALL subir_salario_departamento(2,5);  
select * from trabajadores;
```

Funciones definidas por el usuario

- **Devuelve un único valor** como resultado.
- **Admite parámetros de entrada**, pero **no** tiene parámetros de salida (a diferencia de los procedimientos).
- Se puede utilizar en cualquier lugar donde MySQL permita una expresión (por ejemplo, en un SELECT o en una cláusula WHERE).
- Siempre debe contener una cláusula RETURN.

Parámetros

- RETURNS: tipo de dato que devuelve la función.
- DETERMINISTIC / NOT DETERMINISTIC: indica si la función siempre devuelve el mismo valor para los mismos argumentos (útil para optimización).

```
DELIMITER //
```

```
CREATE FUNCTION nombre_funcion(parametro TIPO)
```

```
RETURNS TIPO_RETORNO
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE resultado TIPO_RETORNO;
```

```
    -- Lógica de cálculo
```

```
    RETURN resultado;
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Funciones definidas por el usuario

Las funciones devuelven **un único valor** y pueden usarse dentro de cualquier consulta.

No pueden ejecutar INSERT/UPDATE/DELETE.

DETERMINISTIC: Siempre da el mismo resultado con la misma entrada (para almacenarla en caché). NO DETERMINISTIC cuando depende de valores RAND o fecha.

```
-- ejemplo funciones
DELIMITER //
CREATE FUNCTION salario_con_bonus(sal DECIMAL(8,2), antig INT)
RETURNS DECIMAL(8,2)
DETERMINISTIC
BEGIN
    IF antig >= 10 THEN RETURN sal * 1.10;
    ELSEIF antig >= 5 THEN RETURN sal * 1.05;
    ELSE RETURN sal;
    END IF;
END;
//
DELIMITER ;
SELECT nombre, salario, antigüedad,
        salario_con_bonus(salario, antigüedad) AS salario_final
FROM trabajadores;
```

	nombre	salario	antigüedad	salario_final
►	Rojo Iglesias, Marta	63000.00	12	69300.00
	Gomez Corachán, Manuel	22000.00	15	24200.00
	Perez Carrillo, Iván	50400.00	5	52920.00
	Torres Marqués, Fernando	57750.00	11	63525.00
	Rubio Sánchez, María	36000.00	4	36000.00
	Llamas Rocasolano, Isabel	28000.00	13	30800.00
	Roca Benítez, Elena	35000.00	6	36750.00
	Perez Cano, Isabel	36000.00	2	36000.00
	Nualart Vives, Carlos	30000.00	10	33000.00
	Muñoz Casas, Montse	40000.00	7	42000.00

Los cursores permiten recorrer los resultados **fila por fila** de un SELECT, útil cuando la lógica no puede resolverse únicamente con SQL. MySQL permite el uso de cursores **únicamente dentro de procedimientos almacenados**.

Elementos:

- DECLARE ... CURSOR FOR: asocia el cursor a una consulta SQL.
- OPEN: inicia el cursor y lo prepara para recorrer los resultados.
- FETCH INTO: extrae la siguiente fila del cursor y asigna sus valores a variables.
- NOT FOUND handler: detecta cuándo se ha alcanzado el final del conjunto de resultados.
- CLOSE: libera los recursos asociados al cursor.

```
DECLARE nombre_cursor CURSOR FOR consulta_select;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET variable_control = valor;

OPEN nombre_cursor;

cursor_loop: LOOP
    FETCH nombre_cursor INTO variable1, variable2, ...;
    IF variable_control THEN
        LEAVE cursor_loop;
    END IF;
    -- Acciones a realizar por fila
END LOOP;

CLOSE nombre_cursor;
```


Cursores

```
-- ejemplo cursor
DELIMITER //
CREATE PROCEDURE trabajadores_por_ciudad(IN p_ciudad VARCHAR(40))
BEGIN
    DECLARE v_nombre VARCHAR(50);
    DECLARE v_antig INT;
    DECLARE v_sal DECIMAL(8,2);
    DECLARE fin_cursor BOOL DEFAULT FALSE;

    -- Cursor para seleccionar trabajadores de una ciudad concreta
    DECLARE cur CURSOR FOR
        SELECT nombre, antigüedad, salario
        FROM trabajadores WHERE ciudad = p_ciudad;
    -- Handler para detectar que ya no hay más filas
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = TRUE;

    OPEN cur;
    bucle: LOOP
        FETCH cur INTO v_nombre, v_antig, v_sal;
        IF fin_cursor THEN LEAVE bucle;
        END IF;

        SELECT v_nombre AS nombre, v_antig AS antigüedad,
            v_sal AS salario;
    END LOOP;
    CLOSE cur;
END;
//
DELIMITER ;
```

173 • CALL trabajadores_por_ciudad('Madrid');

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	nombre	antigüedad	salario
▶	Gomez Corachán, Manuel	15	22000.00

Excepciones

En la programación de bases de datos con MySQL, el control de excepciones permite **gestionar situaciones anómalas o imprevistas durante la ejecución** de procedimientos y bloques de código.

MySQL implementa este control mediante los **manejadores de errores** (HANDLER), que actúan ante ciertos eventos definidos, como el final de un cursor o una violación de restricción.

```
DECLARE tipo_manejador HANDLER  
FOR condición  
instrucción_o_bloque;
```

```
DELIMITER //  
CREATE PROCEDURE insertar_cliente_unico(nombre VARCHAR(50))  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        SELECT 'Error al insertar el cliente. Verifique si ya existe.' AS mensaje_error;  
    END;  
  
    INSERT INTO clientes(nombre) VALUES (nombre);  
END;  
//  
DELIMITER ;
```

Disparadores (Triggers)

Un disparador (*trigger*) en MySQL es un objeto de base de datos que ejecuta automáticamente un bloque de instrucciones cuando ocurre un evento (INSERT, UPDATE, DELETE) sobre una tabla específica. Se utiliza para mantener la integridad, registrar auditorías o automatizar procesos derivados del cambio de datos.

Los disparadores se ejecutan de forma implícita por el sistema, no necesitan ser llamados manualmente.

Características

- Solo pueden asociarse a **una tabla concreta**.
- Se ejecutan **antes** o **después** de un evento (BEFORE o AFTER).
- El evento puede ser: INSERT, UPDATE, o DELETE.
- En MySQL, por cada combinación de BEFORE/AFTER y INSERT/UPDATE/DELETE solo puede haber **un disparador por tabla**.

Uso de las variables OLD y NEW

Estas variables permiten acceder a los valores anteriores y posteriores del registro:

- OLD: representa el valor **antes** del cambio (solo en UPDATE y DELETE).
- NEW: representa el valor **después** del cambio (solo en INSERT y UPDATE).

Sintaxis básica:

```
DELIMITER //
```

```
CREATE TRIGGER nombre_disparador
```

```
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
```

```
ON nombre_tabla
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    -- Instrucciones SQL
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Disparadores (Triggers)

Ejemplo: Un disparador (*trigger*) que guarda en una tabla en forma de log todas las modificaciones que se realizan en el salario de los trabajadores.

```
CREATE TABLE trabajadores_log (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  dni VARCHAR(9),  
  nombre VARCHAR(50),  
  salario_anterior DECIMAL(8,2),  
  salario_nuevo DECIMAL(8,2),  
  fecha DATETIME  
);
```

```
-- ejecución del trigger  
UPDATE trabajadores  
SET salario = salario + 2000  
WHERE dni = '11112222A';  
  
select * from trabajadores_log;
```

```
DELIMITER //  
CREATE TRIGGER trg_salario_log  
AFTER UPDATE ON trabajadores  
FOR EACH ROW  
BEGIN  
  IF OLD.salario <> NEW.salario THEN  
    INSERT INTO trabajadores_log  
    VALUES ( NULL, OLD.dni, OLD.nombre, OLD.salario, NEW.salario, NOW() );  
  END IF;  
END;  
//  
DELIMITER ;
```

	id	dni	nombre	salario_anterior	salario_nuevo	fecha
▶	1	11112222A	Rojo Iglesias, Marta	60000.00	62000.00	2025-11-20 07:02:08

07

Bases de datos no relacionales



Las bases de datos no relacionales, comúnmente denominadas bases de datos NoSQL, constituyen una alternativa al modelo relacional tradicional. Surgieron como respuesta a las limitaciones de las bases de datos relacionales para gestionar grandes volúmenes de datos no estructurados o semiestructurados, típicos de entornos distribuidos, aplicaciones web de alto rendimiento y tecnologías emergentes como el Big Data o el Internet de las Cosas (IoT).

Características

- Ausencia de esquema fijo (Schema-less): No requieren una estructura predefinida y cada documento puede tener campos diferentes.
- Flexibilidad: Permite documentos anidados.
- Escalabilidad horizontal: Distribución de carga, escalado mediante nodos. Alta disponibilidad.
- Rendimiento optimizado: Diseñadas para lectura y escritura rápidas.
- Modelo Base: Priorizan disponibilidad
- Adaptación a nuevas necesidades: Big Data, IOT, procesamiento paralelo.

- Orientadas a documentos
 - Formatos: JSON, BSON, XML. Estructuras flexibles y jerárquicas.
 - Casos de uso: catálogos, perfiles, contenidos web. Ejemplos: MongoDB, CouchDB, RavenDB.
- Clave-Valor
 - Tabla de pares clave y valor. Consultas extremadamente rápidas.
 - Usos: sesiones, cachés, tokens. Ejemplos: Redis, DynamoDB, Riak KV.
- Orientadas a columnas
 - Almacenamiento por columnas. Gran eficiencia en consultas analíticas.
 - Usos: Big Data, informes masivos. Ejemplos: Cassandra, HBase, ClickHouse.
- Orientadas a grafos
 - Modelan nodos y relaciones. Usos: redes sociales, recomendaciones, fraude.
 - Ejemplos: Neo4j, ArangoDB, OrientDB.

[Download MongoDB Community Server | MongoDB](#)

Version

8.2.2 (current)

▼

Platform

Windows x64

▼

Package

msi

▼

Download 

 Copy link

More Options 

MongoDB Compass Download (GUI)

Easily explore and manipulate your database with Compass, the GUI for MongoDB. Intuitive and flexible, Compass provides detailed schema visualizations, real-time performance metrics, sophisticated querying abilities, and much more.

Please note that MongoDB Compass comes in three versions: a **full version** with all features, a **read-only version** without write or delete capabilities, and an **isolated edition**, whose sole network connection is to the MongoDB instance.

For more information, see our [documentation pages](#).

Compass

The full version of MongoDB Compass, with all features and capabilities.

Readonly Edition

This version is limited strictly to read operations, with all write and delete capabilities removed.

Isolated Edition

This version disables all network connections except the connection to the MongoDB instance.

Learn more

Version

1.48.2 (Stable)

▼

Platform

Windows 64-bit (10+)

▼

Package

exe

▼

Download 

 Copy link

More Options 

Crear una base de datos

MongoDB organiza los datos utilizando una estructura jerárquica compuesta por:

- **Bases de datos:** Contenedores lógicos independientes, similares a las bases de datos en los SGBD relacionales.
- **Colecciones:** Agrupaciones de documentos dentro de una base de datos. Equivalentes a las tablas relacionales.
- **Documentos:** Unidades básicas de datos, almacenadas en formato BSON (una extensión binaria de JSON).

Este modelo ofrece una gran flexibilidad, ya que no requiere una estructura fija ni el uso de esquemas estrictos. A continuación, se describen los pasos para crear y gestionar bases de datos y colecciones utilizando la shell de MongoDB.

Crear una base de datos

MongoDB no requiere una instrucción explícita de creación de base de datos. Basta con seleccionarla mediante el comando `use`.

Si no existe, se creará automáticamente cuando se inserte el primer dato.

Sintaxis: `use nombre_base_datos`

Ejemplo: `use biblioteca`

Listar todas las bases de datos existentes:

`show dbs`

```
> use biblioteca
< switched to db biblioteca
```

```
>_MONGOSH
> show dbs;
< admin      40.00 KiB
  ciudades  112.00 KiB
  config     96.00 KiB
  local      88.00 KiB
ciudades>
```

Crear una colección

Una colección es un conjunto de documentos relacionados. Las colecciones se crean automáticamente cuando se inserta un primer documento, o bien manualmente con `createCollection()`.

Creación automática (al insertar):

```
db.libros.insertOne({titulo: "Cien años de soledad", autor: "Gabriel García Márquez"})
```

Este comando:

Crea automáticamente la colección libros si no existe.

Inserta el documento proporcionado.

Crea físicamente la base de datos biblioteca si aún no se ha creado.

Creación explícita: `db.createCollection("autores")`

Crear una colección.

Listar colecciones de la base de datos activa:

show collections

Ver base de datos actualmente activa: db

```
> db.libros.insertOne({titulo: "Cien años de soledad", autor: "Gabriel García Márquez"}) ;
< {
  acknowledged: true,
  insertedId: ObjectId('692ca08768038c0adc8b9475')
}
> db.createCollection("autores");
< { ok: 1 }
> show collections;
< autores
  libros
biblioteca>
```

Insertar documentos en una colección.

MongoDB permite insertar datos mediante los métodos `insertOne()` y `insertMany()`.

```
db.usuarios.insertOne({
  nombre: "Laura", edad: 25, ciudad: "Madrid" });
db.usuarios.insertMany([
  {nombre: "Carlos", edad: 30, ciudad: "Sevilla"},
  {nombre: "Ana", edad: 22, ciudad: "Valencia"}
]);
```

```
> db.usuarios.insertOne({
  nombre: "Laura", edad: 25, ciudad: "Madrid" });
db.usuarios.insertMany([
  {nombre: "Carlos", edad: 30, ciudad: "Sevilla"},
  {nombre: "Ana", edad: 22, ciudad: "Valencia"}
]);
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('692dc996d699eeee11219c05'),
    '1': ObjectId('692dc996d699eeee11219c06')
  }
}
```

test> |

Leer documentos de una colección.

MongoDB permite leer datos mediante los métodos `find()` y `find().pretty()`. En compass tienen el mismo resultado. Para visualizar determinados campos ponemos un 1 o un 0 (no visible).

```
> db.usuarios.find();
< {
  _id: ObjectId('692dc996d699eeee11219c04'),
  nombre: 'Laura',
  edad: 25,
  ciudad: 'Madrid'
}
{
  _id: ObjectId('692dc996d699eeee11219c05'),
  nombre: 'Carlos',
  edad: 30,
  ciudad: 'Sevilla'
}
{
  _id: ObjectId('692dc996d699eeee11219c06'),
  nombre: 'Ana',
  edad: 22,
  ciudad: 'Valencia'
}
```

```
> db.usuarios.find({}, {nombre: 1, edad: 1, ciudad: 1, _id: 0});
< {
  nombre: 'Laura',
  edad: 25,
  ciudad: 'Madrid'
}
{
  nombre: 'Carlos',
  edad: 30,
  ciudad: 'Sevilla'
}
{
  nombre: 'Ana',
  edad: 22,
  ciudad: 'Valencia'
}
```


Ordenar documentos de una colección.

Puedes ordenar los resultados de un find() usando el método .sort().

db.coleccion.find(<filtro>).sort({ campo: orden })

- 1 orden **ascendente** (de menor a mayor)
- -1 orden **descendente** (de mayor a menor)

Ejemplos:

db.usuarios.find().sort({nombre:-1});

db.usuarios.find().sort({edad:1});

```
> db.usuarios.find().sort({edad:1});
< {
  _id: ObjectId('692dc996d699eeee11219c06'),
  nombre: 'Ana',
  edad: 22,
  ciudad: 'Valencia'
}
{
  _id: ObjectId('692dc996d699eeee11219c04'),
  nombre: 'Laura',
  edad: 25,
  ciudad: 'Madrid'
}
{
  _id: ObjectId('692dc996d699eeee11219c05'),
  nombre: 'Carlos',
  edad: 30,
  ciudad: 'Sevilla'
}
```

Filtrar documentos en una colección.

En MongoDB se filtra usando el método `find()`, donde el primer parámetro es un objeto que describe las condiciones que los documentos deben cumplir. Las condiciones pueden ser simples (campo = valor) o compuestas usando operadores como `$gt`, `$lt`, `$in`, `$and`, `$or`.

Operador	Significado	Ejemplo
<code>\$eq</code>	Igual a	<code>{ edad: { \$eq: 25 } }</code>
<code>\$ne</code>	Distinto de	<code>{ estado: { \$ne: "activo" } }</code>
<code>\$gt</code>	Mayor que	<code>{ edad: { \$gt: 18 } }</code>
<code>\$gte</code>	Mayor o igual	<code>{ precio: { \$gte: 100 } }</code>
<code>\$lt</code>	Menor que	<code>{ edad: { \$lt: 30 } }</code>
<code>\$lte</code>	Menor o igual	<code>{ fecha: { \$lte: fechaalta("2024-01-01") } }</code>
<code>\$in</code>	Está en un conjunto	<code>{ pais: { \$in: ["ES", "FR", "AR"] } }</code>
<code>\$nin</code>	No está en un conjunto	<code>{ rol: { \$nin: ["admin", "root"] } }</code>

Filtrar documentos en una colección.

Ejemplos:

```
db.usuarios.find({ciudad:"Madrid"});
```

```
db.usuarios.find({edad:{$lt: 30 }});
```

```
> db.usuarios.find({ciudad:"Madrid"});
< {
  _id: ObjectId('692dc996d699eeee11219c04'),
  nombre: 'Laura',
  edad: 25,
  ciudad: 'Madrid'
}
> db.usuarios.find({edad:{$lt: 30 }});
< {
  _id: ObjectId('692dc996d699eeee11219c04'),
  nombre: 'Laura',
  edad: 25,
  ciudad: 'Madrid'
}
{
  _id: ObjectId('692dc996d699eeee11219c06'),
  nombre: 'Ana',
  edad: 22,
  ciudad: 'Valencia'
}
test> |
```

Actualizar documentos en una colección.

Actualizar documentos (Update) Se pueden modificar documentos mediante `updateOne()` o `updateMany()`

```
db.usuarios.updateOne( {nombre: "Laura"}, {$set: {edad: 26}});  
db.usuarios.updateMany( {ciudad: "Madrid"}, {$set: {activo: true}});
```

```
> db.usuarios.updateOne( {nombre: "Laura"}, {$set: {edad: 26}});  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

```
> db.usuarios.find();  
< {  
  _id: ObjectId('692dc996d699eeee11219c04'),  
  nombre: 'Laura',  
  edad: 26,  
  ciudad: 'Madrid',  
  activo: true  
}  
{  
  _id: ObjectId('692dc996d699eeee11219c05'),  
  nombre: 'Carlos',  
  edad: 30,  
  ciudad: 'Sevilla'  
}
```

Eliminar documentos en una colección.

Se pueden borrar documentos con `deleteOne()` o `deleteMany()`.

```
db.usuarios.deleteOne({nombre: "Laura"});  
db.usuarios.deleteMany({edad: {$lt: 25}});
```

```
> db.usuarios.deleteOne({nombre: "Laura"});  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
> db.usuarios.deleteMany({edad: {$lt: 25}});  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
test>
```

```
> db.usuarios.find();  
< {  
  _id: ObjectId('692dc996d699eeee11219c05'),  
  nombre: 'Carlos',  
  edad: 30,  
  ciudad: 'Sevilla'  
}  
test>
```

Agrupamientos en MongoDB.

Para hacer agrupamientos en MongoDB usamos el método `aggregate()` con el operador `$group`. En `$group`, el campo `_id` indica por qué atributo agrupamos y el resto de campos aplican operaciones como suma, contar o promedio.

Los agrupamientos permiten resumir datos igual que en SQL con GROUP BY, pero MongoDB permite encadenar muchas etapas para transformar y analizar la información de forma más flexible.

El operador **\$sum** permite contar documentos cuando se usa como `$sum: 1` o sumar valores de un campo cuando se usa como `$sum: "$campo"`. Para obtener estadísticas, se emplean **\$avg** para calcular promedios, **\$min** para obtener el valor mínimo y **\$max** para el valor máximo dentro de cada grupo.

Agrupamientos en MongoDB.

Ejemplo con una colección de trabajadores.

```
db.trabajadores.insertMany([
  { nombre: "Ana López", ciudad: "Madrid", salario: 2500, departamento: "Ventas" },
  { nombre: "Carlos Ruiz", ciudad: "Barcelona", salario: 2700, departamento: "Ventas" },
  { nombre: "María Gómez", ciudad: "Valencia", salario: 3200, departamento: "Marketing" },
  { nombre: "Javier Santos", ciudad: "Sevilla", salario: 3100, departamento: "Marketing" },
  { nombre: "Lucía Pérez", ciudad: "Bilbao", salario: 2600, departamento: "Ventas" }
]);

db.trabajadores.insertMany([
  { nombre: "Pedro Martín", ciudad: "Madrid", salario: 2800, departamento: "Ventas" },
  { nombre: "Sara Núñez", ciudad: "Barcelona", salario: 2950, departamento: "Marketing" },
  { nombre: "Elena Torres", ciudad: "Valencia", salario: 3300, departamento: "Ventas" }
]);
```

Agrupamientos en MongoDB.

```
> db.trabajadores.aggregate([
  { $group: { _id: "$ciudad", totalTrabajadores: { $sum: 1 } } }]);
< {
  _id: 'Madrid',
  totalTrabajadores: 2
}
{
  _id: 'Sevilla',
  totalTrabajadores: 1
}
{
  _id: 'Bilbao',
  totalTrabajadores: 1
}
{
  _id: 'Barcelona',
  totalTrabajadores: 2
}
{
  _id: 'Valencia',
  totalTrabajadores: 2
}
}
test>
```

```
db.trabajadores.aggregate([
  { $group: { _id: "$ciudad", totalTrabajadores: { $sum: 1 } } }]);
```

```
db.trabajadores.aggregate([
  { $group: { _id: "$departamento", salarioMedio: { $avg:
"$salario" } } } ]]);
```

```
> db.trabajadores.aggregate([
  { $group: { _id: "$departamento", salarioMedio: { $avg: "$salario" } } } ]]);
< {
  _id: 'Ventas',
  salarioMedio: 2780
}
{
  _id: 'Marketing',
  salarioMedio: 3083.3333333333335
}
```


¡Gracias!

A large, vibrant pink abstract graphic on the right side of the slide. It consists of a rounded rectangular shape at the top and a large, sweeping diagonal line that curves upwards and to the right, creating a sense of movement and modern design.